

OS1 kolokvijumi

kategorisani zadaci sa kolokvijuma 2006-2026 sa stranice predmeta

pripremili Luka Simić i Aleksa Marković

Maj 2026

Sadržaj

Predgovor	2
Ulaz/izlaz	3
1. zadatak, treći kolokvijum, avgust 2024.	3
Ulaz/izlaz (blokovski uređaji)	5
3. zadatak, kolokvijum, februar 2026.	5
3. zadatak, kolokvijum, oktobar 1 2025.	7
3. zadatak, kolokvijum, oktobar 2 2025.	7
3. zadatak, kolokvijum, oktobar 3 2025.	9
3. zadatak, kolokvijum, septembar 1 2025.	10
3. zadatak, kolokvijum, septembar 2 2025.	11
Komandna linija	14
4. zadatak, kolokvijum, septembar 1 2025.	14
Fajl sistem (interfejs)	15
4. zadatak, kolokvijum, oktobar 1 2025.	15
4. zadatak, kolokvijum, oktobar 2 2025.	15
4. zadatak, kolokvijum, septembar 2 2025.	16
Fajl sistem (implementacija)	18
4. zadatak, kolokvijum, februar 2026.	18
4. zadatak, kolokvijum, oktobar 3 2025.	18
2. zadatak, treći kolokvijum, avgust 2024.	19
3. zadatak, treći kolokvijum, avgust 2024.	21

Predgovor

Svrha ove zbirke jeste da objedini sve do sada dostupne kolokvijume iz Operativnih sistema 1 iz nekoliko razloga:

- Na kolokvijumima iz OS1 je dozvoljena literatura, i studenti često pristupaju tome tako što odštampaju kolokvijume i njihova rešenja pa se na kolokvijumu snalaze kroz taj odštampani materijal ako vide neki sličan zadatak. Ovom zbirkom postiže se organizacija takvih materijala radi lakšeg snalaženja na kolokvijumu i ušteda u papirima potrebnim za štampanje svih tih rokova.
- Zadaci su kategorisani po oblastima i sličnosti kako bi se lakše vežbali određeni tipovi zadataka na kolokvijumu.

Poglavlja zbirke idu istim redom kao kolokvijumsko gradivo na predmetu, konkretno:

1. Kompajler/linker, stranična organizacija, kontinualna alokacija, segmentna organizacija, segmentno-stranična organizacija, dinamičko učitavanje, preklopi
2. Prekidi, sistemski pozivi, interfejs niti, promena konteksta, sinhronizacija procesa, baferi, proizvođač/potrošač
3. Ulaz/izlaz, komandna linija, fajl sistem

Raspored ovih oblasti po kolokvijumima se retko menja, ali se takva situacija može desiti (usled predviđenih ili nepredviđenih okolnosti). U tom slučaju, proverite koje oblasti dolaze na kolokvijumu sa predmetnim asistentima. Bitno je napomenuti da se ovaj redosled u prošlosti menjao, pa ako se neki zadatak ranije pojavljivao na prvom kolokvijumu to ne mora da znači da je ta oblast i tekuće godine na prvom kolokvijumu. U suštini, **da li neki zadatak može doći na kolokvijumu određujete prvenstveno na osnovu toga iz koje je oblasti, a ne na osnovu toga na kom se kolokvijumu pojavio u prošlosti.**

Greške u formatiranju i kategorizaciji su sigurno prisutne. Ukoliko ih uočite, možete se javiti jednom od autora ili poslati *pull request* na repozitorijum projekta: <https://github.com/siwiki/OS-kolokvijumi>. Ispravljeni dokumenti će biti dostupni u [Releases](#). Svaka pomoć je dobrodošla.

Srećno na kolokvijumu.

Autori

Ulaz/izlaz

1. zadatak, treći kolokvijum, avgust 2024.

- [Postavka](#)
- [Rešenje](#)

Zahtev za operacijom sa nekog ulaznog uređaja predstavljen je strukturom `IRequest`. Date su deklaracije pokazivača preko kojih se može pristupiti registrima kontrolera ovog uređaja, kao i simboličkih konstanti za vrednosti u njima, sa značenjem objašnjenim na predavanjima. Uređaj generiše prekid za svaki spreman podatak, s tim što se taj prekid može maskirati i demaskirati funkcijama čiji su potpisi dati dole.

Uređaj svoje podatke isporučuje u naletima koji pristižu u nepredvidivim intervalima. Kada dođe jedan nalet neodređene dužine (tj. promenljivog broja pojedinačnih podataka), uređaj vrlo brzo isporučuje jedan po jedan podatak, da bi onda napravio pauzu nepredvidivog trajanja, nakon koje opet stiže nov nalet podataka, itd. sve dok se ne završi prenos celog niza podataka iz jednog zahteva. Zato ulaznu operaciju treba obavljati na sledeći način. Uređaj treba prozivati (polling), u očekivanju da naredni podatak stigne brzo nakon prethodnog. Međutim, kako se može dogoditi da je jedan nalet završen i da nastupa pauza, ovo prozivanje treba raditi najviše `MAX_POLL_CNT` puta. Ako se podatak ne pojavi nakon `MAX_POLL_CNT` ispitivanja bita spremnosti, tok kontrole koji izvršava prozivanje treba suspendovati („uspavati“), dozvoliti prekid uređaju kojim će on onda signalizirati spremnost narednog podatka kojim počinje nov nalet, a koji onda ponovo treba učitavati prozivanjem itd. Za uspavljivanje koristiti klasu `Event` koja realizuje događaj sa uobičajenim operacijama `wait` i `signal` koje se mogu pozivati iz prekidnih rutina i odgovarajućim konstruktorom.

```
typedef volatile unsigned int REG;
REG* ioCtrl = ...; // control register
REG* ioStatus = ...; // status register
REG* ioData = ...; // data register

const REG C_START = 0x...;
const REG C_STOP = 0x...;
const REG C_READY = 0x...;

void mask_intr();
void unmask_intr();

struct IRequest {
    REG* buffer; // Data buffer
    size_t len; // Size of the buffer
    void (*signal)(); // Call back upon I/O completion
    IRequest* next; // Next in the list
};
```

Potrebno je implementirati operaciju `transfer`, zajedno sa odgovarajućom prekidnom rutinom `device_ready` za prekid od uređaja, za obradu jednog zahteva.

```
void transfer (IRequest* req);
interrupt void device_ready ();
```

Rešenje

```
IRequest* pending_req;
size_t pending_cur;
REG* pending_buf = 0;
Event sleep(0);

void transfer (IRequest* req) {
    pending_req = req;
    pending_cur = 0;
    pending_buf = (REG*)(pending_req->buffer);
```

```
mask_intr();
*r_control = C_START;

while (pending_cur < pending_req->len;) {
    unsigned long cnt = 0;
    bool ready = false;
    for (; !ready && cnt < MAX_POLL_CNT; cnt++)
        ready = (*r_status) & C_READY;
    if (ready)
        pending_buf[pending_cur++] = *r_data;
    else {
        unmask_intr();
        sleep.wait();
    }
}

*r_control = C_STOP;
pending_req->signal();
}

interrupt void device_ready () {
    pending_buf[pending_cur++] = *r_data;
    mask_intr();
    sleep.signal();
}
```

Ulaz/izlaz (blokovski uređaji)

3. zadatak, kolokvijum, februar 2026.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu implementira se keš blokova sa blokovskih uređaja kodom koji je dat u nastavku. Za svaki uređaj sa datim identifikatorom pravi se jedan objekat klase `BlockIOCache`, inicijalizovan tim identifikatorom, koji predstavlja keš blokova sa tog uređaja. Keš je kapaciteta `CACHESIZE` blokova veličine `BLKSIZE`. Keš je interno organizovan kao heš mapa `map` sa `MAPSIZE` ulaza. Svaki ulaz niza `map` sadrži glavu liste keširanih blokova koji se preslikavaju u taj ulaz. Funkcija `hash` je heš funkcija koja preslikava broj bloka u ulaz u nizu `map`. Glava liste, kao i pokazivač na sledeći element u listi čuvaju se kao indeksi elementa niza `entries` koji sadrži keširane blokove; vrednost -1 označava kraj (null). Svaki element niza `entries` je struktura tipa `CacheEntry` u kojoj je polje `blkNo` broj bloka koji je keširan u tom elementu, polje `next` ukazuje na sledeći element liste u istom ulazu, a polje `buf` je sadržaj samog bloka.

Na početku složene operacije sa uređajem, kod koji koristi keš najpre zahteva da potrební blok bude učitán pozivom funkcije `getBlock` koja vraća pokazivač na niz bajtova u baferu – učitánom bloku. Pošto više ovakvih složenih operacija može biti pokrenuto uporedo, blok iz keša može biti izbačen (zamenjen drugim) samo ako ga više niko ne koristi, što se realizuje brojanjem referenci u polju `refCounter` strukture `CacheEntry`. Prikazana je implementacija funkcije `getBlock` koja treba da obezbedi da je traženi blok u kešu, odnosno učita ga ako nije.

Potrebno je implementirati pomoćnu funkciju `getFreeEntry` koja treba da vrati indeks slobodnog ulaza u nizu `entries` u koji se može učitati traženi blok u keš. Inicijalno je keš prazan i svi ulazi u njemu su slobodni. Ova funkcija treba redom da zauzima elemente niza `entries`, sve dok ima slobodnih. Kada slobodnih ulaza više nema, ona treba da izbaci blok (snimi ga na disk) u prvom ulazu koji je na redu po LRU redosledu (least recently used, najdavnije korišćen), ali samo pod uslovom da se blok u tom ulazu ne koristi (tj. njegov `refCounter` je nula). Ako to nije zadovoljeno, treba da proba sa sledećim skorijim u listi svih korišćenih ulaza uređenoj hronološki. Ako nema mesta u kešu jer nijedan blok ne može da se izbaci, treba vratiti -1. Ukoliko je potrebno dodati ili izmeniti članove ove klase, precizno navesti kako to treba uraditi. Na raspolaganju je i funkcija koja učitava blok, odnosno upisuje blok na dati uređaj:

```
void ioRead (int device, BlkNo blk, Byte* buffer);
void ioWrite(int device, BlkNo blk, Byte* buffer);

typedef unsigned char Byte;    // Unit of memory
typedef long BlkNo;           // Device block number
const unsigned BLKSIZE = ...; // Block size in Bytes

class BlockIOCache {
public:
    BlockIOCache (int device);
    Byte* getBlock (BlkNo blk);
    ...
protected:
    static int hash (BlkNo);
    int getFreeBlock ();
    void pullEntry (int hand);
private:
    static const unsigned CACHESIZE = ...; // Cache size in blocks
    static const unsigned MAPSIZE = ...;  // Hash map size in entries

    struct CacheEntry {
        BlkNo blkNo;
        int next, lruNext, lruPrev, refCounter;
        Byte buf[BLKSIZE];
    };
};
```

```

    int dev;
    int map[MAPSIZE]; // Hash map
    CacheEntry entries[CACHESIZE]; // Cache
    int numOfBlocks = 0; // Number of used entries
    int lruHead = -1, lruTail = -1; // Head, tail of the LRU list
    ...
};

// Pull the given entry to the top of the LRU list (the most recently used)
void BlockIOCache::pullEntry (int hand) {
    if (lruHead==hand) return;
    CacheEntry& entry = entries[hand];
    if (entry.lruPrev!=-1)
        entries[entry.lruPrev].lruNext = entry.lruNext;
    else lruHead = entry.lruNext;
    if (entry.lruNext!=-1)
        entries[entry.lruNext].lruPrev = entry.lruPrev;
    else lruTail = entry.lruPrev;
    entry.lruPrev = -1;
    entry.lruNext = lruHead;
    if (lruHead!=-1) entries[lruHead].prev = hand;
    else lruTail = hand;
    lruHead = hand;
}

Byte* BlockIOCache::getBlock (BlkNo blk) {
    // Find the requested block in the cache and return it if present:
    int entry = hash(blk);
    for (int i=map[entry]; i!=-1; i=entries[i].next)
        if (entries[i].blkNo==blk) {
            pullEntry(i);
            entries[i].refCounter++;
            return entries[i].buf;
        }
    // The block is not in the cache, find a free slot to load it:
    int free = getFreeEntry(entry);
    if (free==-1) return 0; // Error: cannot find space
    // Load the requested block:
    entries[free].blkNo = blk;
    entries[free].refCounter = 1;
    entries[free].next = map[entry];
    map[entry] = free;
    pullEntry(free);
    ioRead(dev,blk,entries[free].buf);
    return entries[free].buf;
}

```

Rešenje

```

int BlockIOCache::getFreeEntry () {
    if (numOfBlocks<CACHESIZE) {
        int hand = numOfBlocks++;
        entries[hand].lruPrev = -1;
        entries[hand].lruNext = lruHead;
        if (lruHead!=-1) entries[lruHead].prev = hand;
        else lruTail = hand;
        lruHead = hand;
        return hand;
    }
}

```

```

    } else
        for (int hand = lruTail; hand != -1; hand = entries[hand].lruPrev) {
            if (entries[hand].refCounter == 0) {
                ioWrite(dev, entries[hand].blkNo, entries[hand].buf);
                return hand;
            }
        }
    return -1;
}

```

3. zadatak, kolokvijum, oktobar 1 2025.

- [Postavka](#)
- [Rešenje](#)

Neki program obilazi i obrađuje jedno ogromno kompletno binarno stablo sa n nivoa i tačno $2^n - 1$ čvorova, tako što obilazak počinje od korena i svaki čvor obilazi i čita samo po jednom. Stablo je zapisano kao niz čvorova poput strukture hipa (heap): koreni čvor je u elementu niza sa indeksom 0, levo dete čvora sa indeksom i je u elementu sa indeksom $2i + 1$, a desno dete je u elementu sa indeksom $2i + 2$. Ovaj niz je redom po indeksima zapisan u fajl koji je smešten na disku sa blokom u koji staju tačno dva susedna elementa niza (elementi 0 i 1 su u jednom bloku, elementi 2 i 3 su u sledećem itd). Ako sistem za svaki fajl kešira uvek jedan i samo jedan blok sa njegovim sadržajem, izračunati koliko je operacija učitavanja bloka sa diska potrebno izvršiti tokom obrade celog ovog stabla ako je obilazak u sledećem redosledu:

1. (5) prefiksno, odozgo nadole po dubini (pre-order top-down depth-first);
2. (5) odozgo nadole po širini (top-down breadth-first).

Odgovor precizno obrazložiti.

Rešenje

1. (5) Čvorovi koji se smeštaju u isti blok su oni sa indeksima $2k$ i $2k + 1$, gde je k neki prirodan broj ili 0. Postoje dve mogućnosti:
 - i) Ova dva čvora pripadaju istom nivou stabla (susedni siblings). Tada ova dva čvora imaju različite roditelje: $k - 1$, odnosno k . Roditelju prvog čvora taj čvor je desno dete, a roditelju drugog čvora taj čvor je levo dete. Kako se deca čvorova uvek obilaze nakon roditelja, posmatrana dva čvora nikada neće biti posećena kao susedna, pa će sa obilazak svakoga morati da se učita blok u kome se taj čvor nalazi. Dakle, za svaki ovakav čvor mora se učitati blok iznova.
 - ii) Prvi čvor je krajnje desni u nivou i , a drugi čvor je prvi levi u nivou $i + 1$. Ni ova dva čvora se nikada ne obilaze jedan za drugim, osim u početnom slučaju kad je $k = 0$, odnosno za koren i njegovo levo dete.

Prema tome, u ovom slučaju samo za čvor sa indeksom 1 neće biti učitani novi blok, za svaki drugi čvor mora se učitati blok. Zato je ukupno potrebno učitati $2^n - 2$ blokova, za $n > 1$, odnosno $\max(2^n - 2, 1)$ blokova.

2. (5) Čvorovi se sada obilaze tačno po redosledu indeksa u nizu, pa će za svaka dva čvora biti potrebno učitati nov blok (plus jedan zbog neparnog broja čvorova). Zato je ukupno potrebno učitati $\frac{2^n - 1 + 1}{2} = 2^{n-1}$ blokova.

3. zadatak, kolokvijum, oktobar 2 2025.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu implementira se keš blokova sa blokovskih uređaja („diskova”). Za svaki uređaj sa datim identifikatorom pravi se jedan objekat klase BlockIOCache, inicijalizovan tim identifikatorom, koji predstavlja keš blokova sa tog uređaja. Keš je interno organizovan kao heš mapa sa MAPSIZE ulaza. Za svaki ulaz i u mapi organizuje se niz $entries[i]$ veličine CACHESIZE za keširane blokove veličine BLKSIZE koji se preslikavaju u taj ulaz i u heš mapi. Svaki element $map[i]$ sadrži broj zauzetih prvih slotova, tj. keširanih blokova u ulazu $entries[i]$. Funkcija hash je heš funkcija koja preslikava broj bloka u ulaz u mapi. Svaki element niza $entries[i][j]$ je struktura tipa CacheEntry u kojoj je polje blkNo broj bloka koji je keširan u tom elementu, a polje buf je sadržaj samog keširanog bloka.

Na početku složene operacije sa uređajem, kod koji koristi keš najpre traži da je potrební blok učitán pozivom funkcije `getBlock` koja vraća pokazivač na niz bajtova u baferu – učitánom bloku. Pošto više ovakvih složenih operacija može biti ugnježdjeno, blok iz keša može biti izbačen (zamenjen drugim) samo ako ga više niko ne koristi, što se realizuje brojanjem referenci u polju `refCounter` strukture `CacheEntry`. Funkcija `evict(i)` izbacuje jedan keširani blok iz punog keša u nizu `entries[i]`, koji može da se izbaci jer nije u upotrebi, ukoliko takav može da nađe, oslobađa njegov ulaz i dekrementira `map[i]`, pa vraća njegov indeks u tom nizu `entries[i]`; ako takav ne može da nađe, ova funkcija vraća `CACHESIZE`.

Implementirati funkciju `getBlock` koja treba da obezbedi da je traženi blok u kešu, odnosno učita ga ako nije. Ako nema mesta u kešu jer nijedan blok ne može da se izbaci, treba vratiti `null`. Ostale članice date klase su implementirane, a na raspolaganju je i funkcija koja učitava blok na dati uređaj:

```
void ioRead(int device, BlkNo blk, Byte* buffer);

typedef unsigned char Byte; // Unit of memory
typedef unsigned long long BlkNo; // Device block number
const unsigned BLKSIZE = ...; // Block size in Bytes

class BlockIOCache {
public:
    BlockIOCache (int device);
    Byte* getBlock (BlkNo blk);
    ...
protected:
    static int hash (BlkNo);
    void evict (int entry);
private:
    static const unsigned CACHESIZE = ...; // Cache entry size in blocks
    static const unsigned MAPSIZE = ...; // Hash map size in entries

    struct CacheEntry { BlkNo blkNo; int refCounter; Byte buf[BLKSIZE]; };

    int dev;
    int map[MAPSIZE]; // Hash map
    CacheEntry entries[MAPSIZE][CACHESIZE]; // Cache
};
```

Rešenje

```
Byte* BlockIOCache::getBlock (BlkNo blk) {
    // Find the requested block in the cache and return it if present:
    int entry = hash(blk);
    for (int i=0; i<map[entry]; i++)
        if (entries[entry][i].blkNo==blk) {
            entries[entry][i].refCounter++;
            return entries[entry][i].buf;
        }
    // The block is not in the cache, find a free slot to load it:
    int free = map[entry];
    if (free==CASHSIZE) free = evict(entry);
    if (free==CASHSIZE) return 0;
    // Load the requested block:
    entries[entry][free].blkNo = blk;
    entries[entry][free].refCounter = 1;
    map[entry].count++;
    ioRead(dev, blk, entries[entry][free].buf);
    return entries[free].buf;
}
```


3. zadatak, kolokvijum, oktobar 3 2025.

- Postavka
- Rešenje

Računar poseduje DMA kontroler koji ima NDMA's nezavisnih kanala koji mogu da obavljaju isto toliko uporednih izlaznih operacija. Date su deklaracije:

```
const int NDMA's = ...;
typedef volatile unsigned int REG;
extern REG dmaCtrl[NDMA's];    // DMA control registers
extern REG dmaStatus[NDMA's];  // DMA status registers
extern REG dmaAddress[NDMA's]; // DMA block address registers
extern REG dmaCount[NDMA's];   // DMA block size registers
```

DMA kontroler poseduje po jedan upravljački, statusni, registar za zadavanje adrese bloka i registar za zadavanje veličine bloka za svaki svoj kanal; registri svake od ovih grupa su redom vezani na susedne adrese, tako da se mogu posmatrati kao niz registara prema gornjim deklaracijama. U upravljačkom registru najniži bit je bit Start kojim se pokreće prenos jednog bloka preko odgovarajućeg DMA kanala, a u statusnom registru najniži bit je bit završetka prenosa (TransferComplete), a bit do njega bit greške (Error). Svi registri su veličine jedne mašinske reči (tip unsigned int). Kada DMA kanal završi zadati prenos, on se automatski zaustavlja (nije ga potrebno zaustavljati upisom u upravljački registar). Završetak prenosa sa bilo kog DMA kanala generiše isti zahtev za prekid procesoru (signali završetka operacije sa svih DMA kanala vezani su na ulazni zahtev za prekid preko OR logičkog kola). Zahtevi za izlaznim operacijama na nekom uređaju na koji se prenos blokova vrši preko bilo kog od ovih DMA kanala vezani su u jednostruko ulančanu listu. Zahtev ima sledeću strukturu:

```
struct IORequest {
    REG* buffer;           // Data buffer (block)
    unsigned int size;     // Buffer (blok) size
    int status;            // Status of operation
    IORequest* next;       // Next in the list
};
```

Na prvi zahtev u listi pokazuje globalni pokazivač ioHead. Kada u praznu listu kernel stavi prvi zahtev, pozvaće operaciju transfer() koja treba da pokrene prenos za taj prvi zahtev na bilo kom trenutno slobodnom DMA kanalu. Kada se završi prenos zadat jednim zahtevom na jednom DMA kanalu, potrebno je u polje status date strukture preneti status završene operacije (0 – ispravno završeno do kraja, -1 – greška) i pokrenuti prenos za sledeći zapis u listi na tom DMA kanalu, i potom izbaciti zahtev iz liste. Obratiti pažnju na to da više DMA kanala mogu završiti prenos i generisati prekid istovremeno. Ako zahteva u listi više nema, ne treba uraditi više ništa (kada bude stavljaao novi zahtev u listu, kernel će proveriti i videti da je ona bila prazna, pa pozvati ponovo operaciju transfer() itd.). Potrebno je napisati kod operacije transfer() i prekidne rutine dmaInterrupt() za prekid od DMA kontrolera. Smatrati da se navedene funkcije izvršavaju izolovano, ne treba brinuti o međusobnom isključenju jer je ono rešeno na višem nivou.

```
void transfer ();
interrupt void dmaInterrupt ();
```

Rešenje

```
IORequest *dmaPending[NDMA's] = {}; // Currently pending requests
```

```
const REG DMA_TRANSFER_COMPLETE = 1, DMA_ERROR = 2, DMA_START = 1;
void startDMA (int i) {
    if (ioHead!=0 && dmaPending[i]==0) {
        dmaPending[i] = ioHead;           // Take the first request,
        ioHead = ioHead->next;            // remove it from the list
        dmaAddress[i] = (REG)dmaPending[i]->buffer; // and assign it to DMA[i]
        dmaCount[i] = (REG)dmaPending[i]->size;
        dmaCtrl[i] = DMA_START;           // Start I/O
    }
}
```

```

void transfer () {
    for (int i=0; i<NDMA; i++) startDMA(i);
}

interrupt void dmaInterrupt () {
    for (int i=0; i<NDMA; i++)
        if (dmaStatus[i] & DMA_TRANSFER_COMPLETE) { // DMA[i] completed
            if (dmaPending[i]==0) continue;           // Exception
            if (dmaStatus[i] & DMA_ERROR)              // Error in I/O
                dmaPending[i]->status = -1;
            else
                dmaPending[i]->status = 0;
            dmaPending[i] = 0;
            startDMA(i);
        }
}
    
```

3. zadatak, kolokvijum, septembar 1 2025.

- [Postavka](#)
- [Rešenje](#)

Dole je dat interfejs drajvera blokovski orijentisanog uređaja („diska”). Drajver obavlja ulazno-izlazne operacije sa diskom pomoću DMA kontrolera. Operacijom setIVTE drajveru se zadaje dodeljen broj ulaza u IVT preko kog će DMA signalizirati završetak zadate operacije; drajver ovaj broj upisuje u odgovarajući registar DMA kontrolera. Operacija sa DMA pokreće se pozivom operacije start sa zadatim brojem bloka na disku blkNo, za prenos bloka podataka na zadatoj adresi buffer i za odgovarajući smer (rdwr, 0 za čitanje, 1 za upis). Nakon završenog prenosa, DMA kontroler generiše prekid sa zadatim brojem ulaza u IVT, a nakon toga status završene operacije može se očitati pozivom operacije getStatus.

Mehanizam prekida omogućava da se u IVT, pored pokazivača na prekidnu rutinu, zada i parametar tipa void* koji odgovara svakom ulazu. Taj parametar se dostavlja prekidnoj rutini prilikom obrade prekida u tom ulazu. Ulaz u IVT postavlja se na zadatu rutinu isr operacijom Interrupts::initIVT.

Implementirati klasu BlockDevice, zajedno sa odgovarajućom prekidnom rutinom, koja uporednim korisničkim procesima pruža usluge prenosa sa blokovskim uređajem. Pri inicijalizaciji, objektu ove klase dostavlja se broj ulaza u IVT koji je dodeljen datom uređaju, kao i drajver tog uređaja. Tada treba inicijalizovati drajver tim brojem ulaza, ali i postaviti vektor u IVT. Procesi zadaju operacije sa diskom pozivom operacija read i write. Ove operacije treba da se izvršavaju jedna po jedna, kako ih procesi zadaju, i da vraćaju status završene operacije. Sinhronizaciju obavljati semaforima koje implementira klasa Semaphore sa interfejsom kao u školskom jezgru.

```

class DeviceDriver {
public:
    virtual void setIVTE(IVTNo ivte);
    virtual void start (BlkNo blkNo, void* buffer, int rdwr);
    virtual int getStatus ();
};

void Interrupts::initIVT (IVTNo ivte, void(*isr)(void*), void*);

class BlockDevice {
public:
    BlockDevice (IVTNo ivte, BlkDeviceDriver* drv);
    int read (BlkNo blkNo, void* buffer);
    int write (BlkNo blkNo, void* buffer);
};

:
    
```

Rešenje

```

class BlockDevice {
public:
    
```

```

BlockDevice (IVTNo ivte, BlkDeviceDriver* drv);

int read (BlkNo blkNo, void* buffer);
int write (BlkNo blkNo, void* buffer);
protected:
    int perform (BlkNo blkNo, void* buffer, int rdwr);
private:
    friend void blkDevISR (void*);
    Semaphore queue, eop;
    BlkDeviceDriver* myDrv;
    static const int RD = 0, WR = 1;
};

inline int BlockDevice::read (BlkNo blkNo, void* buffer) {
    return this->perform(blkNo,buffer,RD);
}

inline int BlockDevice::write (BlkNo blkNo, void* buffer) {
    return this->perform(blkNo,buffer,WR);
}

interrupt void blkDevISR (void* dev) {
    BlockDevice* d = (BlockDevice*)dev;
    d->eop.signal();
}

BlockDevice::BlockDevice (IVTNo ivte, BlkDeviceDriver* drv)
    : queue(1), eop(0), myDrv(drv) {
    Interrupts::initIVT(ivte, blkDevISR, this);
    myDrv->setIVTE(ivte);
}

int BlockDevice::perform (BlkNo blkNo, void* buffer, int rdwr) {
    this->queue.wait();
    myDrv->start(blkNo,buffer,rdwr);
    this->eop.wait();
    int status = myDrv->getStatus();
    this->queue.signal();
    return status;
}

```

3. zadatak, kolokvijum, septembar 2 2025.

- [Postavka](#)
- [Rešenje](#)

Dole je dat interfejs drajvera blokovski orijentisanog uređaja („diska”). Drajver obavlja ulazno-izlazne operacije sa diskom pomoću DMA kontrolera. Operacijom setIVTE drajveru se zadaje dodeljen broj ulaza u IVT preko kog će DMA signalizirati završetak zadate operacije; drajver ovaj broj upisuje u odgovarajući registar DMA kontrolera. Operacija sa DMA pokreće se pozivom operacije start sa zadatim brojem bloka na disku blkNo, za prenos bloka podataka na zadatoj adresi buffer i za odgovarajući smer (rdwr, 0 za čitanje, 1 za upis). Nakon završenog prenosa, DMA kontroler generiše prekid sa zadatim brojem ulaza u IVT, a nakon toga status završene operacije može se očitati pozivom operacije getStatus.

Mehanizam prekida omogućava da se u IVT, pored pokazivača na prekidnu rutinu, zada i parametar tipa void* koji odgovara svakom ulazu. Taj parametar se dostavlja prekidnoj rutini prilikom obrade prekida u tom ulazu. Ulaz u IVT postavlja se na zadatu rutinu isr operacijom Interrupts::initIVT.

Dole je data implementacija klase BlockDevice, zajedno sa odgovarajućom prekidnom rutinom, koja uporednim korisničkim nitima pruža usluge prenosa sa blokovskim uređajem. Pri inicijalizaciji, objektu ove klase dostavlja se broj ulaza u IVT koji je dodeljen datom uređaju, kao i drajver tog uređaja. Niti zadaju operacije sa diskom pozivom operacije perform. Ova operacija obavlja se jedna po jedna, redom kako su korisničke niti nju pozvale.

Prepraviti implementaciju ove klase, bez izmene u interfejsu prema korisničkim nitima, tako da se zahtevi korisničkih

niti stavljaju u ograničeni bafer, a ne odmah sinhrono obrađuju redom. Nit koja je postavila zahtev treba da se suspenduje dok se zahtev ne izvrši. Posebna nit uzima ove zahteve iz bafera jedan po jedan i za njih pokreće operacije sa drajverom uređaja, što omogućava obradu zahteva redosledom koji nije isti kao onaj kojim su u bafer stavljani. Ograničeni bafer implementiran je gotovom šablonskom klasom `BoundedBuffer<typename T, int capacity>`, parametrizovanom tipom elementa koji se stavlja u bafer i njegovim kapacitetom, a čiji je interfejs dat na predavanjima. Sinhronizaciju obavljati semaforima koje implementira klasa `Semaphore` sa interfejsom kao u školskom jezgru, a niti izvođenjem iz klase `Thread` kao u školskom jezgru.

```
class DeviceDriver {
public:
    virtual void setIVTE (IVTNo ivte);
    virtual void start (BlkNo blkNo, void* buffer, int rdwr);
    virtual int  getStatus ();
};

void Interrupts::initIVT (IVTNo ivte, void(*isr)(void*), void*);

class BlockDevice {
public:
    BlockDevice (IVTNo ivte, BlkDeviceDriver* drv);
    static const int RD = 0, WR = 1;
    int perform (BlkNo blkNo, void* buffer, int rdwr);
private:
    friend void blkDevISR (void*);
    Semaphore queue, eop;
    BlkDeviceDriver* myDrv;
};

interrupt void blkDevISR (void* dev) {
    BlockDevice* d = (BlockDevice*)dev;
    d->eop.signal();
}

BlockDevice::BlockDevice (IVTNo ivte, BlkDeviceDriver* drv)
    : queue(1), eop(0), myDrv(drv) {
    Interrupts::initIVT(ivte, blkDevISR, this);
    myDrv->setIVTE(ivte);
}

int BlockDevice::perform (BlkNo blkNo, void* buffer, int rdwr) {
    this->queue.wait();
    myDrv->start(blkNo,buffer,rdwr);
    this->eop.wait();
    int status = myDrv->getStatus();
    this->queue.signal();
    return status;
}
```

Rešenje

```
class BlockDevice : public Thread {
public:
    BlockDevice (IVTNo ivte, BlkDeviceDriver* drv);
    static const int RD = 0, WR = 1;
    int perform (BlkNo blkNo, void* buffer, int rdwr);
protected:
    virtual void run ();
private:
    struct Request {
        BlkNo blkNo; void* buffer; int rdwr; int status; Semaphore* sem;
    };
};
```

```

    BoundedBuffer<Request*,50> buffer;
    BlkDeviceDriver* myDrv;
    Semaphore eop; friend void blkDevISR (void*);
};
inline int BlockDevice::perform (BlkNo blkNo, void* buffer, int rdwr) {
    Semaphore eop = 0;
    Request req{blkNo,buffer,rdwr,0,&eop};
    this->buffer.append(&req);
    eop.wait();
    int status = req.status;
    return status;
}
interrupt void blkDevISR (void* dev) {
    BlockDevice* d = (BlockDevice*)dev;
    d->eop.signal();
}
BlockDevice::BlockDevice (IVTNo ivte, BlkDeviceDriver* drv)
    : myDrv(drv), eop(0) {
    Interrupts::initIVT(ivte, blkDevISR, this);
    myDrv->setIVTE(ivte);
}
void BlockDevice::run () {
    while (true) {
        Request* req = this->buffer.take();
        myDrv->start(req->blkNo,req->buffer,req->rdwr);
        this->eop.wait();
        req->status = myDrv->getStatus();
        req->sem->signal();
    }
}

```

Komandna linija

4. zadatak, kolokvijum, septembar 1 2025.

- [Postavka](#)
- [Rešenje](#)

Dat je podsetnik na neke osnovne Unix komande:

- `cat`: iz svakog fajla koji je naveden kao argument ove komande, redom kojim su oni navedeni, učitava znakove i ispisuje ih na standardni izlaz; ukoliko nema argumenata, znakove učitava sa standardnog ulaza (dok ne naide na znak EOF koji se na konzoli signalizira pritiskom tastera Ctrl+D).
- `ln src_file dst_file`: pravi novu tvrdnu vezu (ulaz u direktorijumu) na postojeći fajl `src_file` u odredištu definisanom sa `dst_file`; opcija `-s` radi isto to, samo što pravi meku (simboličku) vezu.
- `rm`: briše fajl ili direktorijum ili vezu zadat kao argument.

Izvršavaju se sledeće komande po datom redosledu; napisati te komande ili odgovoriti na postavljeno pitanje:

- U postojećem poddirektorijumu `b` tekućeg direktorijuma napraviti nov tekstualni fajl `bdoc` čiji će sadržaj biti isti kao sadržaj postojećeg tekstualnog fajla `adoc` u roditeljskom direktorijumu tekućeg:
- U roditeljskom direktorijumu tekućeg direktorijuma napraviti tvrdnu vezu pod nazivom `hdoc` na fajl napravljen u koraku 1: _____
- U postojećem poddirektorijumu `a` tekućeg direktorijuma napraviti meku vezu pod nazivom `sdoc` na vezu napravljenu u koraku 2: _____
- Obrisati gorepomenute `hdoc` i `adoc`: _____
- Šta će nakon ovoga ispisati komanda `cat a/sdoc`? _____
- Nakon izdate komande `rm b/bdoc`, šta će ispisati komanda `cat a/sdoc`? _____

Rešenje

```
cat ../adoc > b/bdoc
ln b/bdoc ../hdoc
ln -s ../hdoc a/sdoc
rm ../hdoc
rm ../adoc
```

Komanda `cat a/sdoc` će ispisati sadržaj iskopiran u `bdoc` iz obrisanoog fajla `adoc`.

Fajl sistem (interfejs)

4. zadatak, kolokvijum, oktobar 1 2025.

- [Postavka](#)
- [Rešenje](#)

Implementirati funkciju `setAccessBits` koja se koristi u implementaciji sistemskog programa `chmod`; ona postavlja tri bita za prava pristupa do fajla za neku od klasa korisnika (vlasnika, grupu ili ostale). Parametar `oldBits` u najniža tri bita sadrži dosadašnja prava pristupa (r u bitu 2, w u bitu 1 i x u bitu 0). Novopostavljene vrednosti treba upisati u najniža tri bita parametra `newBits`. Parametar `str` sadrži barem jedan i najviše 4 znaka. U `str[0]` je uvek jedan od znakova `=`, `+` ili `-`, sa značenjem kao u sistemskom pozivu `chmod`. U preostala tri znaka mogu (ali ne moraju) da se nađu velika ili mala slova `r`, `w` ili `x` u proizvoljnom redosledu; sve ostale znakove koji se tu eventualno pojave treba ignorisati, a navedeni znakovi smeju i da se ponavljaju.

```
void setAccessBits (const char str[4], short oldBits, short& newBits);
```

Rešenje

```
void setAccessBits (const char str[4], short oldBits, short& newBits) {
    const char mod = str[0];
    newBits = oldBits;
    if (mod=='=') newBits &= ~(short)0b111;
    for (int i=1; i<=3 && str[i]; i++)
        switch (str[i]) {
            case 'r', 'R':
                if (mod=='=' || mod=='+') newBits |= 0b100;
                if (mod=='-') newBits &= 0b011;
                break;
            case 'w', 'W':
                if (mod=='=' || mod=='+') newBits |= 0b010;
                if (mod=='-') newBits &= 0b101;
                break;
            case 'x', 'X':
                if (mod=='=' || mod=='+') newBits |= 0b001;
                if (mod=='-') newBits &= 0b110;
                break;
        }
}
```

4. zadatak, kolokvijum, oktobar 2 2025.

- [Postavka](#)
- [Rešenje](#)

U nekom binarnom fajlu zapisana je ogromna mapa koja preslikava ključeve u vrednosti (oba tipa `long`). Mapa je organizovana kao ulančana lista čvorova tipa `ListNode`. Ovi čvorovi složeni su u sadržaj fajla u proizvoljnom redosledu kao u niz, jedan iza drugog. Prvi element liste je uvek u ulazu 0 ovog niza, a na sledeći element u listi ukazuje polje `next` u čvoru (sadrži indeks sledećeg čvora u listi); vrednost 0 u polju `next` označava kraj liste. Svaki čvor sadrži `count` preslikavanja ključ u vrednost, a najviše njih `MAX_KEYS`. Ključevi su u nizu `keys`, a njihove odgovarajuće vrednosti u koje se preslikavaju u nizu `values`. 1)(5) Korišćenjem POSIX funkcije `read` za čitanje, realizovati funkciju `getNode` koja obezbeđuje da čvor smešten u dati indeks niza bude učitani u memoriju i vraća pokazivač na taj učitani čvor. Parametar `fd` ove funkcije je deskriptor već otvorenog fajla u kom je zapisana mapa. Ignorirati greške, tj. smatrati da su sadržaj fajla i struktura zapisana u njemu ispravni. 2)(5) Korišćenjem realizovane funkcije `getNode`, implementirati funkciju `getValue` koja u opisanoj mapi koja je u već otvorenom fajlu sa deskriptorom `fd` pronalazi dati ključ i vraća vrednost u koju se taj ključ preslikava. Ukoliko ključ ne pronađe, funkcija treba da vrati -1.

```
const short MAX_KEYS = ...;
struct ListNode {
```

```

    long next;
    unsigned short count;
    long keys[MAX_KEYS];
    long values[MAX_KEYS];
};
ListNode* getNode (int fd, long index);
long getValue (int fd, long key);

```

Rešenje

```

#include <unistd.h>

ListNode* getNode (int fd, long index) {
    static ListNode buffer;
    static long loadedNode = -1;
    if (loadedNode!=index) {
        if (read(fd,&buffer,sizeof(ListNode))<sizeof(ListNode))
            handleError("Corrupted file");
        loadedNode = index;
    }
    return &buffer;
}

long getValue (int fd, long key) {
    long index = 0;
    do {
        ListNode* node = getNode(index);
        for (short i=0; i<node->count; i++)
            if (node->keys[i]==key)
                return node->values[i]; // Key found
        index = node->next;
    } while (index);
    return -1; // Key not found
}

```

4. zadatak, kolokvijum, septembar 2 2025.

- [Postavka](#)
- [Rešenje](#)

Fajl sistem FAT16 koji je koristio MS DOS svaki ulaz u direktorijumu predstavlja strukturom FATDirectoryEntry koja zauzima tačno 32 bajta i u kojoj su upisani atributi fajla ili poddirektorijuma. Spisak elemenata direktorijuma zapisuje se kao sadržaj bilo kog fajla. Ovaj spisak je prost niz ovakvih struktura koje se redom dodaju u niz kako se elementi dodaju u direktorijum, pri čemu se za obrisane tj. slobodne ulaze prvi bajt ove strukture postavlja na posebnu vrednost 0xE5. (Ova struktura zapravo predstavlja FCB koji je ugrađen u sam zapis sadržaja direktorijuma.) Jedan bit u polju attr ukazuje na to da li ulaz predstavlja fajl ili poddirektorijum. Polje firstCluster u ovoj strukturi ukazuje na prvi blok sa zapisom sadržaja fajla odnosno poddirektorijuma.

```

typedef struct {
    char    name[8];           // File name (padded with spaces)
    char    ext[3];           // File extension (padded with spaces)
    uint8_t attr;             // File attributes
    uint8_t reserved;         // Reserved (used for Windows NT or VFAT)
    uint8_t creationTimeTenth; // Creation time in tenths of a second
    uint16_t creationTime;    // Creation time
    uint16_t creationDate;    // Creation date
    uint16_t lastAccessDate;   // Last access date
    uint16_t highCluster;     // Unused; always 0 for FAT16
    uint16_t writeTime;       // Last write time
    uint16_t writeDate;       // Last write date
    uint16_t firstCluster;    // First cluster
}

```



```
uint32_t fileSize;           // File size in bytes
} FATDirectoryEntry;
```

Prostor na particiji organizuje se u tri dela:

- na početku particije je FAT koji zauzima određen broj blokova;
- iza toga se nalazi zapis sadržaja korenog direktorijuma koji čini niz od 512 struktura FATDirectoryEntry, pa zauzima određen fiksni broj blokova;
- iza toga se nalaze blokovi u koje se smešta sadržaj fajlova i direktorijuma.

Ako se pretpostavi da je ceo FAT učitani i keširan u operativnoj memoriji, a da osim njega nijedan više blok korenog direktorijuma ili sadržaja fajlova nije učitani (keširan), kao i da svi pomenuti direktorijumi i fajlovi postoje, koliko blokova sa diska mora da bude učitano da bi se odredio broj prvog bloka sa sadržajem fajla file.txt (data staza sadrži n direktorijuma)? Precizno obrazložiti odgovor.

`\dir_1\dir_2\ ... \dir_n\file.txt`

a)(5) ako se pretpostavi da su svi navedeni direktorijumi i dati fajl jedini elementi svojih roditeljskih direktorijuma i nalaze se u njegovom prvom ulazu: Odgovor: _____

Objašnjenje:

b)(5) ako se pretpostavi da su svi navedeni direktorijumi i dati fajl smešteni u 20. ulaz svojih roditeljskih direktorijuma, a blok je veličine 512 bajtova (klaster je jedan blok): Odgovor: _____

Objašnjenje:

Rešenje

1. (5) $n+1$. Objašnjenje: najpre se učitava prvi blok zapisa korenog direktorijuma sa unapred poznate fiksne lokacije. U njegovom prvom ulazu pronalazi se FCB za `dir_1` i tu se nalazi broj prvog bloka zapisa njegovog sadržaja. Zatim se učitava taj blok i u njegovom prvom ulazu određuje broj prvog bloka zapisa sadržaja `dir_2` itd. U n -tom učitanoj bloku pronalazi se broj prvog bloka zapisa sadržaja `dir_n`. U tom, $n+1$ -om učitanoj bloku, u prvom ulazu, pronalazi se broj prvog bloka sadržaja fajla.
2. $2(n+1)$, jer se za svaki element staze mora učitati dva bloka, pošto se on nalazi u drugom bloku sadržaja (jedan blok sadrži 16 ulaza), a pretraga ulaza je sekvencijalna po nizu ulaza, dok se pozicija drugog bloka sa sadržajem dobija iz FAT koji je keširan.

Fajl sistem (implementacija)

4. zadatak, kolokvijum, februar 2026.

- [Postavka](#)
- [Rešenje](#)

U nekom fajl sistemu blok je veličine 512 bajtova, a broj bloka je 32-bitan. Primenjuje se kombinovani indeksirani pritup alokaciji blokova. FCB zauzima jedan blok i sadrži 120 direktnih ulaza sa brojevima blokova sa sadržajem, kao i jedan single indirect i jedan double indirect ulaz. Popuniti sledeću tabelu upisujući ukupan broj alociranih indeksnih blokova, kao i blokova sa podacima za fajl sa sadržajem maksimalne veličine u ovom sistemu. Brojeve pisati kao stepene dvojke ukoliko je tako preglednije.

Nivo indeksa	Broj alociranih blokova za indekse	Broj alociranih blokova za sadržaj
Direct	1 (za sam FCB)	
Single indirect		
Double indirect		

Rešenje

Nivo indeksa	Broj alociranih blokova za indekse	Broj alociranih blokova za sadržaj
Direct	1 (za sam FCB)	120
Single indirect	1	$128 = 2^7$
Double indirect	$1 + 128 = 129$	2^{14}
Ukupno	131	$2^{14} + 2^7 + 120$

4. zadatak, kolokvijum, oktobar 3 2025.

- [Postavka](#)
- [Rešenje](#)

Fajl sistem FAT16 koji je koristio MS DOS svaki ulaz u direktorijumu predstavlja strukturom FATDirEntry koja zauzima tačno 32 bajta i u kojoj su upisani atributi fajla ili poddirektorijuma. (Ova struktura zapravo predstavlja FCB koji je ugrađen u sam zapis sadržaja direktorijuma.) Spisak ulaza u direktorijumu zapisuje se kao sadržaj bilo kog fajla. Ovaj spisak je prost niz ovakvih struktura koje se redom dodaju u niz kako se elementi dodaju u direktorijum, pri čemu se za obrisane tj. slobodne ulaze prvi bajt ove strukture postavlja na posebnu vrednost 0xE5. Spisak se završava ili kada se u FAT lancu naiđe na kraj (vrednost 0 za klaster) ili prvi bajt ulaza ima vrednost 0. Naziv ulaza ima uvek 8 znakova, a ekstenzija 3 znaka (dopunjeni su blanko znakovima ako su kraći) i ovi stringovi se ne završavaju znakom '\0'. Jedan bit u polju attr ukazuje na to da li ulaz predstavlja fajl ili poddirektorijum. Polje firstCluster u ovoj strukturi ukazuje na prvi klaster sa zapisom sadržaja fajla odnosno poddirektorijuma. Klaster na disku je uvek veličine 512 bajtova (jedan blok).

```
typedef struct {
    char    name[8];           // File name (padded with spaces)
    char    ext[3];            // File extension (padded with spaces)
    uint8_t attr;              // File attributes
    uint8_t reserved;          // Reserved (used for Windows NT or VFAT)
    uint8_t creationTimeTenth; // Creation time in tenths of a second
    uint16_t creationTime;     // Creation time
    uint16_t creationDate;     // Creation date
    uint16_t lastAccessDate;   // Last access date
    uint16_t highCluster;      // Unused; always 0 for FAT16
    uint16_t writeTime;        // Last write time
    uint16_t writeDate;        // Last write date
    uint16_t firstCluster;     // First cluster
    uint32_t fileSize;         // File size in bytes
}
```

```

} FATDirEntry;

void* FSCache::getCluster(uint16_t cluster);
uint16_t FAT::getNextCluster(uint16_t cluster);

FATDirEntry* getEntry(FATDirEntry* dir, const char name[], const char ext[]);

```

Statička funkcija `FAT::getNextCluster` vraća broj sledećeg klastera u listi unutar FAT za zadati klaster, a funkcija `FSCache::getCluster` obezbeđuje da je zadati klaster učitao u operativnu memoriju i vraća pokazivač na njega. Realizovati funkciju `getEntry` koja za zadati FCB koji sigurno predstavlja direktorijum (ne treba proveravati, već je provereno) pronalazi ulaz sa datim imenom i ekstenzijom, ukoliko postoji, i vraća pokazivač na taj ulaz (FCB); ako ne postoji, vraća `null`.

Rešenje

```

const int FNAME_LEN = 8, FEXT_LEN = 3;
const int NUM_DENTRIES = 512/32;
const char DENTRY_EMPTY = '\0x05';

int strcmp (const char* str1, const char* str2, int len) {
    for (int i=0; i<len; i++)
        if (*str1++ != *str2++) return 0;
    return 1;
}

FATDirEntry* getEntry (FATDirEntry* dir,
                      const char name[FNAME_LEN], const char ext[FEXT_LEN]) {
    uint16_t cluster = dir->firstCluster;
    while (cluster) {
        FATDirEntry* entry = (FATDirEntry*)FSCache::getCluster(cluster);
        for (int i=0; i<NUM_DENTRIES && entry->name[0]; i++, entry++) {
            if (entry->name[0] != DENTRY_EMPTY &&
                strcmp(entry->name, name, FNAME_LEN) &&
                strcmp(entry->ext, ext, FEXT_LEN)) return entry;
        }
        cluster = FAT::getNextCluster(cluster);
    }
    return 0;
}

```

2. zadatak, treći kolokvijum, avgust 2024.

- [Postavka](#)
- [Rešenje](#)

Tekući direktorijum Unix interpretera komandne linije je inicijalno prazan. U komandnoj liniji se redom zadaju navedene komande. U datim poredima između nekih komandi napisati šta će biti izlaz nakon izvršene komande navedene neposredno ispred tog poreda. Objašnjenje komandi:

- `mkdir directory`: pravi navedeni direktorijum sa podrazumevanim pravima pristupa; pretpostaviti da su ta podrazumevana prava pristupa dovoljna da tekući korisnik uradi sve komande
- `cat`: ukoliko nije naveden ulazni fajl, ovaj sistemski program sadržaj koji daje na svoj standardni izlaz uzima sa svog standardnog ulaza; pretpostaviti da je nakon prve ove komande korisnik uneo znakove `xyz` sa tastature, kao i kontrolni znak za kraj ulaza
- `ln original_filename link_name`: pravi tvrdi vezu `link_name` ka fajlu `original_filename`
- `rm node`: uklanja čvor

```

mkdir a
ls
-----
cat >b

```

```
ls
-----
cat b
-----
mkdir c
ls
-----
ln b a/d
ls
-----
cat b>a/e
ls a
-----
ln a/e c/f
cat c/f
-----
rm b
cat a/d
-----
ls
-----
rm c/f
ls a
-----
cat a/e
-----
```

Rešenje

```
mkdir a
ls
a
cat >b
ls
a b
cat b
xyz
mkdir c
ls
a b c
ln b a/d
ls
a b c
cat b>a/e
ls a
d e
ln a/e c/f
cat c/f
xyz
rm b
cat a/d
xyz
ls
a c

rm c/f
ls a
d e
```

```
cat a/e
xyz
```

3. zadatak, treći kolokvijum, avgust 2024.

- [Postavka](#)
- [Rešenje](#)

Neki fajl sistem primenjuje kombinovanu tehniku indeksirane alokacije sadržaja fajla. U FCB fajla polje `singleIndex` predstavlja direktni indeks kao niz od `SingleIndexSize` elemenata, pri čemu svaki element sadrži broj fizičkog bloka sa sadržajem fajla (ili 0, ako je neiskorišćen). Ako veličina sadržaja fajla preraste veličinu podržanu ovim indeksom, naredni blokovi sadržaja fajla (preko ove veličine) indeksiraju se indeksom u dva nivoa. Za te potrebe, polje `dblIndex` u FCB sadrži broj bloka na disku u kom je indeks drugog nivoa. Taj blok sa indeksom drugog nivoa sadrži `DblIndexSize` ulaza sa brojevima blokova sa sadržajem fajla, odnosno 0 ako ulaz nije iskorišćen. Polje `size` u FCB sadrži veličinu fajla u bajtovima. Veličina bloka je `BlockSize`. Svi ulazi u indeksu prvog nivoa i indeksu drugog nivoa (ako postoji) su sigurno postavljeni na validne vrednosti (broj bloka sa sadržajem ili 0).

Realizovati funkciju `truncateFile` koja treba da obriše (odseče) sadržaj iza zadate pozicije fajla čiji je FCB dat, tako da veličina sadržaja tog fajla postane `newSize` bajtova (ovaj argument sigurno ima ispravnu vrednost koja je manja ili jednaka prethodnoj veličini fajla):

```
void truncateFile (FCB* fcb, size_t newSize);
```

Na raspolaganju je funkcija koja vraća pokazivač na sadržaj bloka na disku sa datim brojem, učitano u keš:

```
void* getBlock (PBlock blkNo);
```

i funkcija koja blok sa datim brojem označava slobodnim:

```
void freeBlock (PBlock blkNo);
```

Rešenje

```
void truncateFile (FCB* fcb, size_t newSize) {
    fcb->size = newSize;
    size_t blk = (newSize > 0) ? ((newSize - 1) / BlockSize + 1) : 0;
    for (; blk < SingleIndexSize; blk++)
        if (fcb->singleIndex[blk]) {
            freeBlock(fcb->singleIndex[blk]);
            fcb->singleIndex[blk] = 0;
        }

    if (fcb->dblIndex) {
        PBlock* dblIx = (PBlock*)getBlock(fcb->dblIndex);
        bool dblIxEmpty = false;
        for (blk -= SingleIndexSize, dblIxEmpty = (blk == 0); blk < DblIndexSize; blk++)
            if (dblIx[blk]) freeBlock(dblIx[blk]);
        if (dblIxEmpty) {
            freeBlock(fcb->dblIndex);
            fcb->dblIndex = 0;
        }
    }
}
```